

个人作业报告

题目：**大数据原理与技术**

作业六：图文多模态问答

姓名	林宏宇
学号	23320093
院系	计算机学院
专业	计算机科学与技术
指导教师	王昌栋

【摘　要】

在学习王老师的大数据原理与技术课程[1]后，在此报告中完成相应的课后作业，本次作业（作业-6）任务为图文多模态问答（VQA），要求使用 ViLT 预训练模型（通过 Hugging Face 库）进行调用与测试，可进行微调或直接推理。

实验要求：

- 使用 ViLT 预训练模型（Hugging Face）。
- 尝试调用模型，输入图片与问题（例如“图中有什么动物?”），也可自定义问题，并输出模型答案。

作业提交及注意事项：

- 提交内容：代码与简要报告（报告需包含测试案例截图，至少 2 页）。
- 命名格式：学号 + 姓名 + 作业 6.zip
- 提交邮箱：BDSysu2026Spring@163.com
- 截止时间：布置作业之后的下一个周日 24:00
- 严禁抄袭：任何一次抄袭，平时成绩按零分处理

本实验首先使用 Hugging Face 提供的 ViLT 预训练模型及其处理器对输入图像和问题进行前向推理。对 CIFAR-10 子集进行快速验证时，图像先被转换为 PIL.Image 并按模型要求缩放与归一化（如 Resize 到模型输入尺寸），问题使用通用模板（例如 “What is in the image?”）或自定义问题。推理得到的自由文本答案被保存为原始输出，随后对结果做小写化、去除标点、同义词替换与基于相似度的模糊匹配，将开放域答案映射到 CIFAR-10 的 10 类标签。作为基线比较，使用预训练的 ResNet-18 对目标类别进行监督微调（测试子集约 1000 张，batch=64，epochs=3，lr=1e-3）。

结论：预训练的 ViLT 在开放域 VQA 上能给出有意义的自由文本答案，但在将自由文本映射到固定标签集合时存在误匹配风险。本实验表明，针对目标类别进行监督微调（例如 ResNet-18）或结合候选重排序与同义词映射的后处理策略，可显著提高分类准确率与一致性。

关键词： 图文多模态问答，VQA，ViLT，Hugging Face

目录

1 ViLT 模型简介 1

2 图文多模态问答（VQA）任务实验 2
2.1 实验目标 2
2.2 方法概述 2
2.3 数据与预处理 2
2.4 实验设置 3
2.5 实验结果 4
2.6 实验结果分析 5

3 结论 6
3.1 实验结论 6
3.2 心得体会 6

参考文献 7

附录 A 附录：实现代码 8

1 ViLT 模型简介

ViLT（Vision-and-Language Transformer）是一类将图像和文本在同一 Transformer 架构中联合建模的视觉—语言模型，其实现要点包括：

- 文本嵌入：使用词表嵌入、位置嵌入与分段（token type）嵌入对输入问题进行编码；嵌入维度通常为 768。
- 视觉嵌入：将图像划分为若干 patch 并通过一个小的卷积投影层（例如 kernel=32, stride=32）将每个 patch 投影为与文本相同维度的视觉 token。
- 联合编码器：文本 token 与视觉 token 在同一序列中送入多层 Transformer 编码器（如 12 层、每层包含自注意力与前馈网络），以捕捉跨模态交互。
- 池化与分类头：编码后通过池化器得到整体表征，随后接入小型 MLP/LayerNorm/GELU 层并映射到输出词表上（在本实验使用的预训练模型，其分类器最终输出维度约为 3129，用于生成开放式答案的 logits）。

上述设计的直接后果是：模型在生成答案时使用的是一个大规模的开放域词表，这有利于开放式问答的表达能力，但在面对固定、有限的标签集合时，会增加后处理映射的难度（需要把自由文本答案可靠地映射到预定义类）。

2 图文多模态问答（VQA）任务实验

2.1 实验目标

本章报告基于视觉—语言模型的图文多模态问答（VQA）实验，并辅以监督的图像分类微调作为基线比较。具体目标为：

- 验证基于预训练视觉—语言模型的 VQA 推理流程并保存推理输出与评估指标；
- 设计并评估把模型自由文本答案映射到固定类标签的后处理策略（精确匹配、模糊匹配与同义词映射）；
- 对图像做监督微调（ResNet-18），作为强监督基线并对比不同方法的准确率与误判模式；
- 分析误判类型并提出可行的改进路径（例如 CLIP 打分、候选重排序或针对性微调）。

2.2 方法概述

本实验采取两条并行技术路线：

1) VQA 推理：使用现成的视觉—语言模型对图像与问题对进行前向推理，得到自由文本答案并通过后处理映射为类别标签；
2) 监督分类：以预训练的图像分类网络为初始化，对目标类别做小规模微调，得到一个可直接输出目标标签的基线分类器。

后处理映射的核心思想是：先对预测文本做规范化（小写、去除标点），做精确匹配；若失败则尝试规则同义词替换（如”plane”→”airplane”、”car”→”automobile”等），再使用基于相似度的模糊匹配做候选匹配；若仍无法匹配则将该输出标为未对齐以便人工分析。

2.3 数据与预处理

2.3.1 数据集说明

本实验主要采用公开的小型图像分类数据集作为图像来源（本次实验以 CIFAR-10 为例）。数据集说明如下：

- 数据规模：数据集中通常包含 60000 张 32×32 彩色图像，划分为训练集与测试集。
- 类别与分布：共有 10 个类别（例如 airplane、automobile、bird、cat、deer、dog、frog、horse、ship、truck），各类别在标准划分中近似均衡（测试集中约 1000 张/类）。
- 图像特性：图像分辨率较低，物体通常占据较小像素区域且存在遮挡或复杂背景；这对基于 patch 的视觉 token 表征与小物体识别提出挑战，尤其在将图像上采样以适配预训练视觉—语言模型时需格外注意空间信息的丢失或插值失真。
- 金标与缺失：实验评估以数据集提供的类别标签作为金标；若某些推理样本缺少金标或无法确定正确类别，则在评估时排除这些样本以避免对准确率产生偏差。
- 子集与快速验证：为便于快速迭代和调试，可使用训练/测试子集进行小规模验证，完整评估则需使用标准的测试拆分以保证统计稳健性。
- 对 VQA 的影响：由于预训练视觉—语言模型通常对输入图像做 patch 投影（按固定 kernel 与 stride），输入分辨率和缩放策略会直接影响视觉 token 的空间粒度；实验中对图像做了适当的缩放以保证视觉编码具有足够的空间信息，从而提高跨模态交互的效果。

2.3.2 预处理

- 对用于视觉—语言模型的输入，统一把图像转换为 PIL.Image，并交由模型的处理器做缩放与归一化；
- 对于监督分类器，按预训练权重的输入分布进行数据增强与归一化（例如 Resize 到 224、随机水平翻转、ImageNet 均值/方差归一化）；

2.4 实验设置

2.4.1 VQA 推理

使用的预训练视觉—语言模型在开放域 VQA 上表现良好，推理环节的要点如下：

- 问题模板：采用通用问题”What is in the image?”（针对性实验可替换为”What animal is in the image?”）；
- 输出诊断：在推理前后打印并检查模型词表与典型预测，以帮助调试映射策

略；

- 结果记录：对每条样本保存原始预测文本、映射后的类别与原始标签。

2.4.2 监督微调（ResNet-18）

监督微调用于得到可比较的有监督基线，训练要点：

- 网络结构：以 ResNet-18 初始化，替换最后的全连接层以适配目标类别数；
- 超参数（实验示例）：小规模验证配置使用少量样本（如 2000 条训练 / 1000 条测试）、batch=64、epochs=3、学习率 =1e-3；输入按预训练权重做 Resize/Normalize。训练后保存模型权重与训练曲线以供分析。

2.5 实验结果

2.5.1 三组训练策略准确率对比

为清晰展示不同训练策略的效果，下面给出三组实验的准确率对比：

实验	准确率
未训练（初始权重）	0.6064
小规模微调	0.6790
中等规模微调	0.8500

表 2.1 三种训练策略的准确率比较

2.5.2 未训练 ViLT 每类准确率

下面给出 ResNet-18 在小规模微调测试拆分上的每类统计：

类别	总数	正确数	准确率
airplane	103	71	0.6893203
automobile	89	87	0.9775280
bird	100	87	0.8700
cat	103	56	0.5436893
deer	90	50	0.5555555
dog	86	24	0.2790697
frog	112	87	0.7767857
horse	102	74	0.7254901
ship	106	79	0.7452830
truck	109	64	0.5871559

表 2.2 ResNet-18 小规模微调在测试拆分上的每类统计

2.6 实验结果分析

2.6.1 总体表现

本节汇总本次实验的关键数值与总体观察：

- ResNet-18 未训练基线在评估拆分上的准确率为 0.606430155210643。
- ResNet-18 小规模微调（train[:2000]）在测试拆分上的准确率为 0.6790。
- 中等规模微调（train[:10000]）在验证集上的最佳验证准确率为 0.8500。

2.6.2 每类性能与混淆概况

从小规模微调的每类统计可以看出明显的类别间差异（表格中已有详细数值）：

- ‘automobile‘ 准确率最高（≈0.9775），说明该类别的外观特征对模型较友好；
- ‘dog‘ 准确率最低（≈0.2791），表明该类在小样本情况下具有高内在变异，易与 ‘cat‘ 混淆；
- ‘frog‘、‘ship‘、‘horse‘ 等类别处于中等偏上水平（约 0.72–0.78 区间）。

2.6.3 混淆矩阵揭示的典型误判包括

- ‘truck‘ 常被误判为 ‘automobile‘ / ‘bus‘（小样本下类别边界模糊）；
- ‘dog‘ 与 ‘cat‘ 的互相混淆反映了姿态、视角与毛色的多样性带来的识别困难；
- 未训练的 VQA 输出包含大量与目标标签无关的短语（如”pizza”,”keyboard” 等），说明开放域生成与闭集标签对齐存在较大噪声。。

2.6.4 方法比较与深入分析

- 监督微调在闭集分类任务上表现稳健，可通过更多带标签数据或更长训练获得进一步提升；
- VQA（ViLT）适用于开放式问答与富文本解释场景，但直接用于返回闭集标签需要可靠的对齐机制——当前采用的字符串/模糊匹配易受词汇多样性和描述习惯影响；
- 中等规模样本带来的验证精度显著提升（从小样本的 best val≈0.728 到 0.85），表明数据量仍是影响泛化能力的关键因素。

3 结论

3.1 实验结论

- 监督微调（基于 ResNet-18）在固定标签集的闭集分类任务中是最直接且有效的方式：小规模微调已显著优于未经对齐的模型，中等规模训练带来更大的提升（验证精度可达 0.85）。
- 通用 VQA（ViLT）在开放域任务上具有表达与解释优势，但其开放域输出与固定标签集合的对齐能力有限，需要候选重排序或任务特定微调才能稳定匹配闭集标签。
- 类别间差异提醒我们在数据采集与增强上需做有针对性的设计（特别是高内在变异的类别），并可考虑融合跨模态相似度打分以提高映射鲁棒性。

3.2 心得体会

本次实验强化了两点经验：一是“方法要与任务匹配”——开放域的生成模型并不天然适合返回固定标签集的高精度需求；二是“数据与对齐比模型更重要”——在有限模型容量下，合适的数据量、质量与对齐策略通常能带来比更复杂模型更显著的收益。同时学习了 ViLT 的架构与推理流程，理解了视觉—语言模型在实际应用中的优势与局限，为后续的多模态研究打下了基础。

参考文献

[1] 王昌栋. 第一章绪论[Z]. 2026.

附录 A 附录：实现代码

```
import argparse
import json
import time
from pathlib import Path

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
import torchvision.transforms as T
import torchvision.models as models
from datasets import load_dataset


def normalize_image(x):
    return x


class HFDataset(Dataset):
    def __init__(self, hf_ds, transform=None, image_key_candidates=("img", "image", "image_file", "image_path")):
        self.ds = hf_ds
        self.transform = transform
        self.image_key_candidates = image_key_candidates

    def __len__(self):
        return len(self.ds)

    def _get_image(self, item):
        for k in self.image_key_candidates:
            if k in item:
                return item[k]
        for v in item.values():
```

```
            try:
                from PIL import Image
                if isinstance(v, Image.Image):
                    return v
            except Exception:
                pass
        return None

    def __getitem__(self, idx):
        item = self.ds[int(idx)]
        img = self._get_image(item)
        label = item.get("label") if "label" in item else item.get("labels") if "labels" in item else None
        if img is None:
            raise RuntimeError(f"No image found in item keys: {list(item.keys())}")
        if self.transform:
            img = self.transform(img)
        return img, int(label)


def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0
    for xb, yb in loader:
        xb = xb.to(device)
        yb = yb.to(device)
        optimizer.zero_grad()
        out = model(xb)
        loss = criterion(out, yb)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * xb.size(0)
        preds = out.argmax(dim=1)
        correct += (preds == yb).sum().item()
        total += xb.size(0)
```

```
    return running_loss / total, correct / total


def evaluate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for xb, yb in loader:
            xb = xb.to(device)
            yb = yb.to(device)
            out = model(xb)
            loss = criterion(out, yb)
            running_loss += loss.item() * xb.size(0)
            preds = out.argmax(dim=1)
            correct += (preds == yb).sum().item()
            total += xb.size(0)
    return running_loss / total, correct / total


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--train-split", default="train[:2000]")
    parser.add_argument("--test-split", default="test[:1000]")
    parser.add_argument("--model-out", default="models/resnet18_cifar_small.
    pth")
    parser.add_argument("--metrics-out", default="models/
    resnet18_cifar_small_metrics.json")
    parser.add_argument("--batch-size", type=int, default=64)
    parser.add_argument("--epochs", type=int, default=3)
    parser.add_argument("--lr", type=float, default=1e-3)
    args = parser.parse_args()

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    print("Loading dataset...", args.train_split, args.test_split)
    ds_train = load_dataset("cifar10", split=args.train_split)
```

```
    ds_test = load_dataset("cifar10", split=args.test_split)

    num_classes = 10

    # transforms for pretrained ResNet
    transform_train = T.Compose([
        T.Resize(224),
        T.RandomHorizontalFlip(),
        T.ToTensor(),
        T.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)),
    ])
    transform_eval = T.Compose([
        T.Resize(224),
        T.ToTensor(),
        T.Normalize(mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)),
    ])

    train_ds = HFDataset(ds_train, transform=transform_train)
    test_ds = HFDataset(ds_test, transform=transform_eval)

    train_loader = DataLoader(train_ds, batch_size=args.batch_size, shuffle=
    True, num_workers=2, pin_memory=(device.type=="cuda"))
    test_loader = DataLoader(test_ds, batch_size=args.batch_size, shuffle=
    False, num_workers=2, pin_memory=(device.type=="cuda"))

    model = models.resnet18(pretrained=True)
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    model = model.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=args.lr)

    best_acc = 0.0
    history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc":
    []}

    for epoch in range(1, args.epochs + 1):
        t0 = time.time()
```

```
        train_loss, train_acc = train_one_epoch(model, train_loader, criterion, optimizer, device)
        val_loss, val_acc = evaluate(model, test_loader, criterion, device)
        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)
        t1 = time.time()
        print(f"Epoch {epoch}/{args.epochs} - train_loss={train_loss:.4f} train_acc={train_acc:.4f} val_loss={val_loss:.4f} val_acc={val_acc:.4f} time={t1-t0:.1f}s")
        if val_acc > best_acc:
            best_acc = val_acc
            Path(args.model_out).parent.mkdir(parents=True, exist_ok=True)
            torch.save({"model_state_dict": model.state_dict(), "num_classes": num_classes}, args.model_out)

    if not Path(args.model_out).exists():
        Path(args.model_out).parent.mkdir(parents=True, exist_ok=True)
        torch.save({"model_state_dict": model.state_dict(), "num_classes": num_classes}, args.model_out)
        print("Untrained model saved to:", args.model_out)


    metrics = {
        "best_val_acc": best_acc,
        "history": history,
        "device": str(device),
        "train_split": args.train_split,
        "test_split": args.test_split,
        "model_out": args.model_out,
    }
    Path(args.metrics_out).parent.mkdir(parents=True, exist_ok=True)
    with open(args.metrics_out, "w", encoding="utf-8") as f:
        json.dump(metrics, f, indent=2)

    print("Training complete. Saved model and metrics:", args.model_out, args.metrics_out)
```

```
172
173
174 if __name__ == '__main__':
175     main()
```